

Fast N-body simulation on Unity compute shader

Roman Boiko

DH2323 Project specification

Introduction

N-body simulations are computer simulations solving the N-body problem. N-body problem consists of calculating positions of a certain number N of bodies which all interact with each other. Analytical solutions only exist for $N=2$ and certain configurations of higher N s. Most N-body problems are solved numerically. Such simulations give insight into astrophysical processes and other natural phenomena.

In this project a particular type of an N-body algorithm is implemented on Unity engine. This project specification was written after the project has been completed and only imitates what the specification could have been in the first weeks.

Description

There are multiple ways to solve N-body problems with approximations, but this project focuses on the all-bodies approach, where every interaction is explicitly calculated, meaning an $O(N^2)$ complexity. The algorithm proposed by the article in GPU Gems 3 splits the $N \times N$ matrix of interactions into tiles, which are then calculated sequentially.¹ This allows to efficiently use GPU memory, while still supporting high degree of parallelism. The goal of the project is to implement it on Unity.

Implementation

Article only describes the kernel which is made for CUDA environment. Project deliverable will have the same kernel but rewritten in Unity and supplemented by other necessary code to actually run animations. Unity has a specific algorithm type called Compute shader which is used for parallel GPU calculations which will certainly be used for the kernel implementation. Aside from kernel itself, a general controller script and a specific shader for rendering a large number of bodies will be made. An integration kernel has to be created as well. Some specific N-body scenario, like two galaxies colliding, can be implemented as well to showcase the simulation working. Some article about galaxy simulations can be used as the basis. Additional refinements, like coloring, can be done later. As Unity uses C# as the major language, project expectedly will be written primarily on it as well.

Evaluation

Since the algorithm is purely mathematical and focuses on performance with no visual output, performance tests are the first things to be done. Effect of different parameters, such as tile size, loop unroll count and number of bodies can be measured, mimicking the original article. After an actual galaxy model is implemented, some astrophysical simulations can be done to check, whether the results agree with previous research. This can help gauge the scientific potential of such an algorithm.

1. Nylons, L., Harris, M. & Prins, J. Fast n-body simulation with cuda. *GPU gems* **3**, 62–66 (2007).